Untitled

65090502417 ปิยพัทธ์ จันทร์ทวี

2025-11-13

```
library(torch)
```

```
## Warning: package 'torch' was built under R version 4.4.3
```

```
library(luz)
library(torchvision)
```

```
## Warning: package 'torchvision' was built under R version 4.4.3
```

```
library(torchdatasets)
```

```
## Warning: package 'torchdatasets' was built under R version 4.4.3
```

```
library(zeallot)
```

```
## Warning: package 'zeallot' was built under R version 4.4.3
```

```
library(ISLR)

max_features <- 10000
maxlen <- 500

imdb_train <- imdb_dataset(root = ".", split = "train", num_words = max_features, maxlen = ma
xlen)
imdb_test  <- imdb_dataset(root = ".", split = "test",  num_words = max_features, maxlen = ma
xlen)

vocab <- c(rep(NA, imdb_train$index_from - 1), imdb_train$get_vocabulary())
tail(names(vocab)[imdb_train[1]$x])
```

```
## [1] "thinks" "what"   "a"      "load"   "of"     "crap"
```

```
model_base <- nn_module(
  "LSTMModel",
  initialize = function() {
    self$embedding <- nn_embedding(num_embeddings = max_features + 3, embedding_dim = 32)
    self$lstm <- nn_lstm(input_size = 32, hidden_size = 32, num_layers = 1, batch_first = TRU
E)
    self$dense <- nn_linear(32, 1)
  },
  forward = function(x) {
    x <- self$embedding(x)
    lstm_out <- self$lstm(x)
    last_output <- lstm_out[[1]][ , -1, ]  # last timestep output
    self$dense(last_output) %>% torch_flatten(start_dim = 1)
  }
)

model_base <- model_base %>%
  setup(
    loss = nn_bce_with_logits_loss(),
    optimizer = optim_rmsprop,
    metrics = list(luz_metric_binary_accuracy_with_logits())
  ) %>%
  set_opt_hparams(lr = 0.05)

fitted_base <- model_base %>% fit(
  imdb_train,
  epochs = 5,
  dataloader_options = list(batch_size = 256),
  valid_data = imdb_test
)

acc_base <- evaluate(fitted_base, imdb_test, dataloader_options = list(batch_size = 512))
print(acc_base)
```

```
## A `luz_module_evaluation`
## ── Results ──────────────────────────────────────────────────────────────────────
## loss: 0.6302
## acc: 0.6651
```

```
# predict
pred_probs <- torch_sigmoid(predict(fitted_base, imdb_test))
# predicted labels
pred_labels <- (pred_probs > 0.5)$to(dtype = torch_int())


true_labels <- torch_tensor(
  sapply(seq_len(length(imdb_test)), function(i) {
    y <- imdb_test[i][[1]][[2]]
    if (y == 1) 0 else 1
  }),
  dtype = torch_int()
)

# create data frame
df_results <- data.frame(
  pred_labels = as.integer(as_array(pred_labels)),
  true_labels = as.integer(as_array(true_labels))
)
```

จากการทำ LSTM (learning rate = 0.05) ได้ว่า มีบทความที่ทำนายผิดจากความเป็นจริง โดยตัวอย่างคือ บทความแรก

” i went and saw this movie last night after being coaxed to by a few friends of mine i’ll admit that i was reluctant to see it because from what i knew of ashton kutcher he was only able to do comedy i was wrong kutcher played the character of jake fischer very well and kevin costner played ben randall with such professionalism the sign of a good movie is that it can toy with our emotions this one did exactly that the entire theater which was sold out was overcome by laughter during the first half of the movie and were moved to tears during the second half while exiting the theater i not only saw many women in tears but many full grown men as well trying desperately not to let anyone see them crying this movie was great and i suggest that you go see it before you judge”

จากการวิเคราะห์บทความ จะสามารถ สังเกตเห็นคำที่ทำให้ model ทำนายผิด โดยจะเป็นคำที่กำกวมหรือสลับอารมณ์ไปมาอย่าง เช่น “i was wrong” ที่ ประโยคนี้จะสามารถอยู่ได้ในทั้งบริบทเชิงบวก หรือ บริบทเชิงลบ (เช่น i was wrong เพราะ ติดว่าหนังจะดี กับ i was wrong เพราะคิว่าหนังจะห่วย) อย่างในบริบทของบทความนี้ การเกริ่นบทความ ด้วยความรู้สึกเชิงลบ จากความคิดที่ว่าหนังจะห่วย จากนั้นก็ใช้คำสลับอารมณ์กับบทความจนกลายเป็นบทความเชิงบวกใน ท้ายที่สุด

เมื่อลองให้ generative ai อย่าง chatgpt ช่วยในการประเมิณอารมณ์บทความ ได้ว่า ai สามารถทำนายอารณ์ได้อย่างแม่นยำและ ยังระบุได้อีกว่าแต่ละช่วงของบทความอารณ์เปลี่ยนแปลงไปอย่างไร

```
library(torch)
library(luz)
library(ggplot2)
library(zeallot)
library(torchdatasets)

maxlen <- 300
num_words <- 10000

imdb_train <- imdb_dataset(root = ".", split = "train", num_words = num_words, maxlen = maxle
n)
imdb_test <- imdb_dataset(root = ".", split = "test", num_words = num_words, maxlen = maxlen)

wc <- sapply(seq_along(imdb_train), function(i) length(imdb_train[i]$x))


model_mod_new <- nn_module(
  initialize = function() {
    self$embedding <- nn_embedding(num_words + 3, 64)
    self$lstm <- nn_lstm(
      input_size = 64,
      hidden_size = 128,
      num_layers = 2,
      batch_first = TRUE,
      bidirectional = TRUE
    )
    self$dense1 <- nn_linear(128 * 2, 64)
    self$act1 <- nn_leaky_relu(negative_slope = 0.01)
    self$drop1 <- nn_dropout(p = 0.3)
    self$dense2 <- nn_linear(64, 32)
    self$act2 <- nn_relu()
    self$output <- nn_linear(32, 1)
  },
  forward = function(x) {
    c(output, c(hn, cn)) %<-% (x %>% self$embedding() %>% self$lstm())
    output[,-1,] %>%
      self$dense1() %>%
      self$act1() %>%
      self$drop1() %>%
      self$dense2() %>%
      self$act2() %>%
      self$output() %>%
      torch_flatten(start_dim = 1)
  }
)

model_mod_new <- model_mod_new %>%
  setup(loss = nn_bce_with_logits_loss(),
        optimizer = optim_rmsprop,
        metrics = list(luz_metric_binary_accuracy_with_logits())) %>%
  set_opt_hparams(lr = 0.01)

fitted_mod_new <- model_mod_new %>% fit(
  imdb_train,
  epochs = 8,
```

```
  dataloader_options = list(batch_size = 128),
  valid_data = imdb_test
)

acc_mod_new <- evaluate(fitted_mod_new, imdb_test, dataloader_options = list(batch_size = 25
6))
```